

R Shiny App Project

Zachary Serby and Illya Vu

##Load the package

```
library(shiny)
library(ggplot2)
library(tidyr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(plotly)
```

```
##
## Attaching package: 'plotly'
```

```
## The following object is masked from 'package:ggplot2':
##
##   last_plot
```

```
## The following object is masked from 'package:stats':
##
##   filter
```

```
## The following object is masked from 'package:graphics':
##
##   layout
```

```
library(shinythemes)
library(tools)
library(DT)
```

```
##
## Attaching package: 'DT'
```

```
## The following objects are masked from 'package:shiny':
##
##   dataTableOutput, renderDataTable
```

```
library(viridis)
```

```
## Loading required package: viridisLite
```

```
library(readr)
song_normalize <- read_csv("/Users/hanhvu/Documents/STA 230/song_normalize.csv")
```

```
## Rows: 2000 Columns: 18
```

```
## — Column specification —————
## Delimiter: ", "
## chr (3): artist, song, genre
## dbl (14): duration_ms, year, popularity, danceability, energy, key, loudness...
## lgl (1): explicit
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Data manipulation

```
#Seperate the Genre column
topsong = song_normalize %>% separate(genre, c("genre1", "genre2"), ", ")
```

```
## Warning: Expected 2 pieces. Additional pieces discarded in 417 rows [6, 16, 22, 28, 36,
## 43, 45, 50, 53, 54, 61, 64, 68, 69, 70, 78, 81, 93, 98, 100, ...].
```

```
## Warning: Expected 2 pieces. Missing pieces filled with `NA` in 721 rows [1, 5, 7, 10,
## 11, 12, 13, 17, 18, 19, 20, 23, 24, 26, 30, 33, 34, 35, 38, 44, ...].
```

```
#Pivot the dataset longer to show two most prominent genres of songs
spotify = pivot_longer(topsong, cols = c(genre1, genre2), names_to = "order",
                        values_to = "genre") %>% na.omit(genre2) %>% select(!order) %>%
  mutate(new_column = ifelse(grepl("easy listening", genre) | grepl("set()", genre), genre, NA)) %>%
  mutate(genre = gsub("easy listening|set()", NA, genre))
```

```
#Clean the data
spotify$genre = toTitleCase(spotify$genre)
spotify$explicit[spotify$explicit == "FALSE"] <- "Not Explicit"
spotify$explicit[spotify$explicit == "TRUE"] <- "Explicit"
colnames(spotify)[colnames(spotify) == "duration_ms"] <- "duration"
```

```

## Set up the UI object
ui <- navbarPage("Spotify 2000s Wrap",
  theme = shinytheme("yeti"), #Select theme for the app
  tabPanel("Top Hits 2000s Overview",
    sidebarLayout(position = "left",
      sidebarPanel(
        #Set up x-variables to select
        selectInput(inputId = 'option', label = "Choose your variable:",
          choices = c("Genre" = "genre",
            "Explicit" = "explicit")),
        #Set up filter by range of year
        sliderInput(inputId = 'years_bar', label = "Year released", 1998, 2020,
          value = c(1998, 2020), sep = ""),
        #Set up filter by artists
        selectizeInput(inputId = 'artist2', label = "Artist",
          choices = spotify$artist, multiple = TRUE)),
    mainPanel(
      tabsetPanel(
        tabPanel("Bar Chart of Top Hits of 2000s",
          h5("The bar chart presents the number of songs of each genre
            and explicitness, with proportions of artists of choice's
            contribution to each category"),
          plotOutput('barchart'),
          wellPanel(
            span("Number of songs selected:",
              tableOutput("song2")))),
        tabPanel("Top Song Rankings",
          h5("Overall ranking of songs"),
          DT::dataTableOutput("poptable"))))),
  tabPanel("Top Hits 2000 Trend",
    sidebarLayout(position = "left",
      sidebarPanel(
        #Set up x-variable to select
        selectInput(inputId = 'music', label = "Choose your variable:",
          choices = c("Tempo" = "tempo",
            "Duration" = "duration",
            "Loudness" = "loudness",
            "Energy" = "energy")),
        #Set up filter by range of year
        sliderInput(inputId = 'years_summary', label = "Year released", 1998, 2020,
          value = c(1998, 2020), sep = ""),
        #Set up filter by artist
        selectizeInput(inputId = 'artist', label = "Artist",
          choices = spotify$artist, multiple = TRUE)),
    mainPanel(
      tabsetPanel(
        tabPanel("Scatter plot of Popularity and Music Traits",
          h5("This scatter plot with prediction line examines
            the correlation between tempo,duration, loudness,
            and energy of each song and its popularity ranked by
            Spotify to reflect the trend in music taste"),
          plotlyOutput("scatterplot")),
        wellPanel(
          span("Number of songs selected:",
            tableOutput("song"))))
      )),
  tabPanel("Top Hits 2000 Explorer",
    sidebarLayout(
      sidebarPanel(
        #Set up x-variables to select
        selectInput('music1', "Choose your variable:",
          choices = c("Tempo" = "tempo", "Duration" = "duration",
            "Loudness" = "loudness", "Energy" = "energy")),
        #Set up four filters
        sliderInput('dance', "Danceability", 0.129, 0.975, value = c(0.129, 0.975)),
        sliderInput('speech', "Speechiness", 0.0232, 0.576, value = c(0.0232, 0.576)),
        sliderInput('instru', "Instrumentalness", 0, 0.985, value = c(0, 0.985)),
        sliderInput('live', "Liveness", 0.0215, 0.853, value = c(0.0215, 0.853))
      ),
    mainPanel("Spotify Explorer",
      h5("Explore your music of the 2000s for dancing, singing, and other activities
        with or without music and ambience noise"),
      plotlyOutput("explot"),
      wellPanel("Data Table", DT::dataTableOutput("table")))),
  tabPanel("About",
    p("Spotify is a popular music streaming service that offers users access to a vast library of songs, albums,
    and playlists from various genres. It allows 345 million monthly active users (as of January 2022) to enjoy music on-demand, create
    personalized playlists, and discover new tracks based on their preferences. Recording users' activities with different titles, Spotify

```

```
API contains millions of data of different songs on their duration, sound, lyrics to evaluate their musical properties and rank their popularity with listeners. Based on this information, this platform has curated a", em("Top Hits of 2000"), "playlist including all the most iconic songs in the 2000s."),
    br(),
    p("This R Shiny App analyzes the musical properties of chosen songs to find the most successful songs and artists, look into the trend of music throughout the year and how it is related to the popularity of music and allow users to explore the 2000s songs best fit for their personal demand."),
    br(),
    p("The musical properties of songs will be consider based on:"),
    p(strong("Tempo (bpm)"), "shows the overall estimated tempo of a track in beats per minute (BPM),"),
    p(strong("Duration (ms)"), "shows the length of the track in milliseconds,"),
    p(strong("Loudness (dB)"), "is the overall loudness of a track in decibels,"),
    p(strong("Energy"), "represents a perceptual measure of intensity and activity,"),
    p(strong("Danceability"), "predicts how likely will people dance to the song,"),
    p(strong("Speechiness"), "reports the presence of spoken words in a track,"),
    p(strong("Instrumentalness"), "predicts whether a track contains any to no vocals,"),
    p(strong("Liveness"), "detects the presence of ambience noise in the recording,"),
    p("and", strong("Popularity"), "reports the Spotify Index Popularity of how popular the songs are relative to all other songs on a 0-to-100 scale. It is calculated by total streams of a song, how recently the song was played and the frequency it has been played. This information is available through the Spotify developers API.")))
```

```
## Warning: The select input "artist2" contains a large number of options;
## consider using server-side selectize for massively improved performance. See
## the Details section of the ?selectizeInput help topic.
```

```
## Warning: The select input "artist" contains a large number of options; consider
## using server-side selectize for massively improved performance. See the Details
## section of the ?selectizeInput help topic.
```

```
## Warning: Navigation containers expect a collection of
## `bslib::nav_panel()`/`shiny::tabPanel()`s and/or
## `bslib::nav_menu()`/`shiny::navbarMenu()`s. Consider using `header` or `footer`
## if you wish to place content above (or below) every panel's contents.
```

```

## Set up the server function
server <- function(input, output) {

  #Output mainPanel page 1 tab 1
  output$barchart <- renderPlot({
    data_to_plot <-
      if (is.null(input$artist2)) { # when no artists were chosen in the filter
        spotify %>%
          filter(!is.na(genre)) %>%
          filter(between(year, input$years_bar[1], input$years_bar[2]))
      }
      else{
        if (input$option == "Genre") { # when Genre is the x-variable
          newartist3 <- reactive({
            req(input$artist2)
            spotify[spotify$artist %in% input$artist2,]
          })
          newartist3() %>%
            filter(!is.na(genre))%>%
            filter(between(year, input$years_bar[1], input$years_bar[2]))
        }
        else{ #other interaction
          newartist2 <- reactive({
            req(input$artist2)
            spotify[spotify$artist %in% input$artist2,]
          })
          newartist2() %>% distinct(song, .keep_all = TRUE) %>%
            filter(!is.na(genre))%>%
            filter(between(year, input$years_bar[1], input$years_bar[2]))
        }
      }

    #Graph the bar chart using ggplot
    ggplot(data_to_plot, aes_string(x = input$option,
                                   fill = if (is.null(input$artist2)) {input$option}
                                   else{"artist"})) +
      geom_bar(position = if (is.null(input$artist2)) {"stack"}
               else{"fill"}) +
      labs(title = "Bar Chart", x = input$option, y = "Count") +
      scale_fill_brewer(palette = "Set3") +
      theme_minimal() +
      theme(legend.position = ifelse(is.null(input$artist2), "none", "right"))
  })

  #Output wellPanel page 1 tab 1
  newartist2 = reactive({spotify[spotify$artist %in% input$artist2,]})
  output$song2 <- renderTable({
    q <- newartist2()%>% distinct(song, .keep_all = TRUE)

    # Use the tapply function to count songs for each artist
    artist_counts2 <- tapply(q$song, q$artist, length)

    # Create a data frame with two columns: Artist and Songs
    artist_counts_df2 <- data.frame(
      Artist = names(artist_counts2),
      Songs = artist_counts2
    )
  })

  #Output mainPanel page 1 tab 2
  output$poptable <- DT::renderDataTable(DT::datatable({

    temp <- spotify %>%
      distinct(song, .keep_all = TRUE) %>%
      select(song, artist, popularity) %>%
      arrange(desc(popularity)) #Set up table

    data <- temp[, c("song","artist", "popularity")]

    data
  })))

  #Output mainPanel page 2
  newartist = reactive({spotify[spotify$artist %in% input$artist,]})
  output$scatterplot <- renderPlotly({
    if (is.null(input$artist)){ # when no artist is chosen in the filter
      b = input$music
      temp = spotify %>% distinct(song, .keep_all = TRUE) %>%
        filter(between(year, input$years_summary[1], input$years_summary[2]))
    }
  })
}

```

```

#Set up regression line
model <- lm(popularity ~ get(b), data = temp)
y.ft <- model$fitted.values

#Graph the scatter plot and regression line in plotly
plot_ly(data = temp) %>%
  add_trace(type = "scatter", mode = "markers", x = ~get(b), y = ~popularity,
            text = ~paste0("Artist:", artist, "<br>", "Year:", year, "<br>", "Song Name:", song ),
            hoverinfo = "text") %>%
  layout(xaxis= list(title = b)) %>%
  add_lines(x = ~get(b), y = y.ft, name = "Prediction line") }

else{ #other interaction

  b = input$music
  n = newartist() %>% distinct(song, .keep_all = TRUE) %>%
    filter(between(year, input$years_summary[1], input$years_summary[2]))

  #Set up the regression line
  model <- lm(popularity ~ get(b), data = n)
  y.ft <- model$fitted.values

  #Graph the scatter plot in plotly
  plot_ly(data = n) %>%
    add_trace(type = "scatter", mode = "markers", x = ~get(b), y = ~popularity, color = ~artist,
              text = ~paste0("Artist:", artist, "<br>", "Year:", year, "<br>", "Song Name:", song ),
              hoverinfo = "text") %>%
    layout(xaxis= list(title = b)) %>%
    add_lines(x = ~get(b), y = y.ft, name = "Prediction line") }
})

#Output wellPanel page 2
newartist = reactive({spotify[spotify$artist %in% input$artist,]})
output$song <- renderTable({
  q <- newartist() %>% distinct(song, .keep_all = TRUE)

  # Use the tapply function to count songs for each artist
  artist_counts <- tapply(q$song, q$artist, length)

  # Create a data frame with two columns: Artist and Songs
  artist_counts_df <- data.frame(
    Artist = names(artist_counts),
    Songs = artist_counts
  )
})

#Output for page 3
output$explot <- renderPlotly({
  c = input$music1
  temp1 = spotify %>% distinct(song, .keep_all = TRUE) %>% #Set up filter
    filter(between(danceability, input$dance[1], input$dance[2])) %>%
    filter(between(speechiness, input$speech[1], input$speech[2])) %>%
    filter(between(instrumentalness, input$instru[1], input$instru[2])) %>%
    filter(between(liveness, input$live[1], input$live[2]))

  #Graph the scatter plot
  plot_ly(data = temp1) %>%
    add_trace(type = "scatter", mode = "markers", x = ~get(c), y = ~popularity, color = ~artist,
              text = ~paste0("Artist:", artist, "<br>", "Year:", year, "<br>", "Song Name:", song ),
              hoverinfo = "text") %>%
    layout(xaxis= list(title = c), showlegend = FALSE) })

#Output wellPanel page 3

output$table <- DT::renderDataTable(DT::datatable({

  b = input$music
  temp = spotify %>% distinct(song, .keep_all = TRUE) %>%
    filter(between(danceability, input$dance[1], input$dance[2])) %>%
    filter(between(speechiness, input$speech[1], input$speech[2])) %>%
    filter(between(instrumentalness, input$instru[1], input$instru[2])) %>%
    filter(between(liveness, input$live[1], input$live[2])) %>%
    select(c("artist", "song", "tempo", "duration", "loudness", "energy"))

  data <- temp[,1:6]

```

```
      data
    )))

}

## Build and run

shinyApp(ui, server)
```

Shiny applications not supported in static R Markdown documents